PROVENDA - SYSTEM MANAGEMENT

EXHAUSTIVE FUNCTIONAL & TECHNICAL REFERENCE MANUAL

Complete documentation of all functions, modules, algorithms and business rules

Application:	Provenda - Integrated Feed Mill & POS Management ERP
System Type:	Real-time Feed Production, Inventory & Multi-tender POS
Reference Currency:	CFA Franc (FCFA / XAF)
Valuation Method:	Weighted Average Cost (WAC / CMP) with attributable landing costs
Architecture & Stack:	React 19 + TypeScript + Tailwind CSS (Vite Engine)
Coverage:	100% of Functions, Calculation Formulas, Models & RBAC Security
Edition Date:	September 25, 2026
Document Status:	Official - Administrator, Auditor & Developer Reference

EXECUTIVE SUMMARY & DOCUMENT STRUCTURE

This official document provides comprehensive documentation for every software function and feature comprising the Provenda application. Every method in the central AppContext state manager, each financial and accounting formula (WAC/CMP, Gross Margins, Physical Inventory Variance, Discount Ceilings), each user interface module (POS, Stock Ledger, Purchases, Invoicing, Customers, Suppliers, Reports, Audit Trail, Settings) and role-based security policies (RBAC) are thoroughly detailed with their validation rules, inputs/outputs and use cases.

- Chapter 1: Global Architecture, Data Models & RBAC Security Matrix (Roles & Permissions)
- Chapter 2: Stock Engine, Unit Conversions & Dynamic WAC / CMP Calculation
- Chapter 3: Point of Sale Terminal (POS), Multi-tender Checkout & Controlled Discounts
- Chapter 4: Supply Chain, Goods Receipt, Landing Costs & Supplier Registry
- Chapter 5: Invoicing, Customer Credit Ledgers, Due Dates & Credit Notes (Avoirs)
- Chapter 6: Physical Inventory Counting Sessions & Stock Variance Reconciliation
- Chapter 7: Financial Reports, Commercial Margins & Security Audit Trail
- Chapter 8: Complete Codebase Function Directory (Names, Signatures, Inputs, Logic)

1. System Architecture & RBAC Security Matrix

Provenda is architected around a unified React Context state engine (AppContext), guaranteeing transactional consistency across inventory levels, checkout registers, purchases, customer receivables and the immutable audit trail. Security is strictly enforced using Role-Based Access Control (RBAC).

1.1 Roles & Privileges Matrix

Role	Discount Limit	POS Checkout	Stock & WAC	Purchases & Receipts	Customers & Debts	Settings & Backups
Administrator	100% max	Full Access	Create, WAC, Adjust.	Create, Validate	Limits, Settle, Credits	Complete Access & Multi-Store
Manager (Depot)	15% max	Full Access	View, Stock Adjust.	Drafts, Validate	Settlement, Exceptions	View & Reports
Cashier	0% (Blocked)	Sales & Checkout	View only	No access	Debt settlement	No access
Stockist / Warehouse	0% (Blocked)	No access	Adjust., Movements, Inv.	Physical goods receipt	No access	No access

1.2 Multi-Store Architecture (Branches)

The application supports managing multiple physical points of sale (e.g. Main Mill, Port Depot, Market Annex). The active store selector dynamically filters sales, stock inventory and receipts. Each employee can be assigned to a dedicated branch, while Administrators can switch branches or view consolidated data anytime.

1.3 Core Entities & Data Models

Entity	Primary Keys	Critical Fields	Business Purpose
Product	id, code (e.g. PRD-001)	baseUnit, saleUnit, conversionFactor, cmp, stockCurrentBaseUnit	Feed mill item managed in kg or bag with calculated WAC.
Invoice	id, number (FAC-YYYY-XXXXXX)	customerId, items[], totalNet, totalCostOfGoodsSold, grossMargin	Sales invoice with itemized margins & tax.
Purchase	id, referenceNumber (ACH-...)	supplierId, items[], transportFees, otherFees, status	Supply order incorporating attributable landing fees.
StockMovement	id, timestamp	productId, type, quantityBaseUnit, costUnitCMP, justification	Immutable ledger of all physical inbound/outbound flows.
InventorySession	id, referenceNumber (INV-...)	items[], theoreticalStock, physicalStock, varianceValue	Physical stocktake session with valuation of discrepancies.
Customer	id, code (CLI-001)	creditLimit, currentDebt, type (farmer, cooperative, reseller)	Client account with strict credit ceiling enforcement.
AuditLog	id, timestamp	userId, action, targetObject, oldValue, newValue, justification	ISO-compliant immutable security log for financial audits.

2. Stock Engine & Weighted Average Cost (WAC / CMP) Algorithm

2.1 Official Mathematical WAC Formula

WEIGHTED AVERAGE COST FORMULA UPON GOODS RECEIPT:

```
New_WAC = ( (Initial_Stock × Old_WAC) + Inbound_Cost ) / ( Initial_Stock + Received_Qty )
```

Where Inbound_Cost = (Received_Qty × Unit_Purchase_Price) + Transport_Fees + Other_Landing_Costs

2.2 Attribution of Landing & Inbound Freight Fees

Under SYSCOHADA and IFRS agricultural accounting standards, acquisition costs for raw feed grains (Corn, Soymeal, Premixes) must include supplier invoice prices PLUS transport, offloading and handling fees. Provenda automatically incorporates these fees proportionally into the received unit cost, ensuring that profit margins and ending inventory valuations are never artificially overstated.

2.3 Unit Conversions (Base Stock Unit vs Commercial Sale Unit)

Packaging Description	Commercial Unit	Stock Base Unit	Conversion Factor	Practical Example
50 kg Bag (Finisher / Layer feed)	bag (sac)	kilogram (kg)	50	Sale of 2 bags = Auto deduction of 100 kg in inventory
25 kg Bag (Starter feed)	bag (sac)	kilogram (kg)	25	Sale of 4 bags = Auto deduction of 100 kg in inventory
Bulk loose retail by the kilo	kilogram	kilogram (kg)	1	Sale of 12.5 kg = Auto deduction of exactly 12.5 kg in inventory
20 Liter Jug (Oil / Molasses)	jug (bidon)	liter (L)	20	Receipt of 5 jugs = Auto increment of 100 L in inventory

3. Point of Sale Terminal (POS) & Checkout Engine

The POS terminal is built for rapid counter operations with barcode/product code search, automatic packaging unit conversions, real-time background margin computation, strict enforcement of discount authorization ceilings based on user role, and multi-tender settlement (Cash, Mobile Money Orange/MTN, Bank Transfer or Customer Credit).

3.1 Real-Time Profitability & Gross Margin Formulas

```
1. Gross Subtotal = Sum ( Quantity × Catalog Unit Price )
2. Net Revenue = Gross Subtotal - Total Approved Discount
3. Cost of Goods Sold (COGS) = Sum ( Base Quantity × Product WAC / CMP )
4. Gross Margin = Net Revenue - COGS  |  Margin Rate (%) = (Gross Margin / Net Revenue) × 100
```

3.2 Commercial Discount Approval Rules

User Profile	Allowed Discount	Action if Exceeded	Required Traceability
Cashier	0% (No discount allowed)	Blocked by system. Requires manager intervention.	Cannot checkout without override
Manager (Depot)	Up to 15% maximum	Over 15% blocked. Requires administrator approval.	Manager name, reason & rate logged on invoice
Administrator	Up to 100% (Full exception)	Allowed with mandatory justification note.	Logged in immutable audit trail with justification

8. Comprehensive Codebase Function Directory (AppContext)

This section provides technical and operational documentation for every method implemented in AppContext.tsx. It details TypeScript signatures, input parameters, return structures, exceptions, and business logic.

Theme, i18n & Navigation

Function	TypeScript Signature	Operational Description & Algorithm
toggleTheme()	() => void	Toggles visual theme between dark and light (#FAF7F2 cream). Persists state in localStorage under provenda_theme.
setLanguage(lang)	(lang: "fr" | "en") => void	Sets global active language. Persists choice in localStorage and dynamically refreshes all interface keys.
t(key)	(key: string) => string	Resolves localized string keys (e.g. "nav.pos", "stock.cmp") with fallback to French if key is missing.

Multi-Store Management (Branches)

Function	TypeScript Signature	Operational Description & Algorithm
setCurrentSalesPointId(id)	(id: string) => void	Switches the active work branch. Dynamically filters sales, purchases and inventories for that branch.
addSalesPoint(sp)	(sp: Omit<SalesPoint, "id"|"createdAt">) => void	Creates a new branch with unique code (e.g. POS-02), address, phone and manager. Logs to audit trail.
updateSalesPoint(sp)	(sp: SalesPoint) => void	Updates details, phone and manager of an existing store location with full audit traceability.
deleteSalesPoint(id)	(id: string) => void	Deletes a secondary branch. The main store (isMain) can never be deleted.

Authentication & Roles (RBAC)

Function	TypeScript Signature	Operational Description & Algorithm
login(email, password)	(email, password) => { success, error }	Authenticates user credentials, verifies active status, updates lastLogin timestamp and creates session.
logout()	() => void	Terminates active session, clears authentication keys and redirects to login view.
switchRole(role)	(role: Role) => void	Allows instant profile switching for testing or demoing permission boundaries (admin, manager, cashier, stockist).
addUser(user)	(user) => { success, error }	Creates an employee user account with assigned role, custom discount limit (0% cashier, 15% manager) and store.
updateUser(user)	(user: User) => void	Updates credentials, role, status or discount limits with comprehensive audit logging.
deactivateUser(id)	(id: string) => void	Deactivates an employee account immediately revoking access while preserving historic transactions.

Supply Chain, Goods Receipt & WAC Engine

Function	TypeScript Signature	Operational Description & Algorithm
createPurchaseDraft(p, spId)	(purchase, targetSalesPointId?) => string	Generates draft purchase order with reference ACH-YYYY-XXXXXX, line items, and estimated landing fees.
validatePurchaseReception(id)	(purchaseId: string) => void	Core WAC engine: recalculates new WAC/CMP per item, increments base unit inventory, and logs inbound movement.
cancelPurchase(id, reason)	(purchaseId: string, reason: string) => void	Cancels draft purchase order with mandatory justification note. Physical stock is untouched.

Sales, POS Checkout, Discounts & Invoicing

Function	TypeScript Signature	Operational Description & Algorithm
createSale(...)	(customerId, items, discount?, payment?, notes?, isCredit?, creditDueDate?, spId?) => { success, invoiceId, error }	Executes atomic sales transaction: checks stock availability, enforces user discount ceiling, validates customer credit limits, computes gross margin, generates FAC invoice, decrements stock at WAC, records payments and logs audit.
cancelInvoiceWithCreditNote(...)	(invoiceId: string, reason: string) => { success, error }	Cancels validated invoice: generates official Credit Note AVO-YYYY-XXXXXX, restocks inventory, decreases customer debt and logs audit.

Function	TypeScript Signature	Operational Description & Algorithm
recordDebtPayment(...)	(invoiceId, amount, method, ref?, notes?) => { success, error }	Records client debt payment: reduces invoice remaining balance and client total debt, marks as paid if balance is 0.

Stock Adjustments & Physical Inventory

Function	TypeScript Signature	Operational Description & Algorithm
recordStockAdjustment(...)	(productId, qtyChangeBaseUnit, type, justification) => { success, error }	Performs manual stock adjustments for spoilage, damage, internal livestock consumption or corrections. Creates movement and audit.
createInventorySession(cat?)	(categoryFilter?: string) => string	Opens physical inventory session INV-YYYY-XXXXXX and snapshots theoretical stock levels for all products.
validateInventorySession(...)	(sessionId, countedItems, notes?) => { success, error }	Reconciles physical count with theoretical stock, calculates quantitative and monetary variances at WAC, updates stock and closes session.

System Backup, Restore & Selectors

Function	TypeScript Signature	Operational Description & Algorithm
exportSystemBackup()	() => string	Exports all application state tables (products, invoices, purchases, movements, customers, suppliers, audit, settings) into a timestamped JSON file.
restoreSystemBackup(json)	(jsonString: string) => { success, message }	Validates JSON integrity, restores entire application state across all modules and records audit action.
resetToDefaultData()	() => void	Resets application to calibrated factory seed data (sample feed products, users, suppliers, customers).
getLowStockProducts()	() => Product[]	Memoized selector returning all active products where current stock is less than or equal to minimum alert threshold.
getOutOfStockProducts()	() => Product[]	Selector returning all active products that are strictly out of stock (<= 0).